PEP 681: Data Class Transforms

Status: Final · Created: 02-Dec-2021 · Public domain / CC0

Abstract

557 introduced the dataclass to the Python stdlib. Several popular libraries have behaviors that are similar to dataclasses, but these behaviors cannot be described using standard type annotations. Such projects include attrs, pydantic, and object relational mapper (ORM) packages such as SQLAlchemy and Django.

Most type checkers, linters and language servers have full support for dataclasses. This proposal aims to generalize this functionality and provide a way for third-party libraries to indicate that certain decorator functions, classes, and metaclasses provide behaviors similar to dataclasses.

These behaviors include:

* Synthesizing an __init__ method based on declared data fields. * Optionally synthesizing __eq__, __ne__, __lt__, __le__, __gt__ and __ge__ methods. * Supporting "frozen" classes, a way to enforce immutability during static type checking. * Supporting "field specifiers", which describe attributes of individual fields that a static type checker must be aware of, such as whether a default value is provided for the field.

The full behavior of the stdlib dataclass is described in the Python documentation.

This proposal does not affect CPython directly except for the addition of a dataclass_transform decorator in typing.py.

Motivation

There is no existing, standard way for libraries with dataclass-like semantics to declare their behavior to type checkers. To work around this limitation, Mypy custom plugins have been developed for many of these libraries, but these plugins don't work with other type checkers, linters or language servers. They are also costly to maintain for library authors, and they require that Python developers know about the existence of these plugins and download and configure them within their environment.

Rationale

The intent of this proposal is not to support every feature of every library with dataclass-like semantics, but rather to make it possible to use the most common features of these libraries in a way that is compatible with static type checking. If a user values these libraries and also values static type checking, they may need to avoid using certain features or make small adjustments to the way they use them. That's already true for the Mypy custom plugins, which don't support every feature of every dataclass-like library.

As new features are added to dataclasses in the future, we intend, when appropriate, to add support for those features on dataclass_transform as well. Keeping these two feature sets in sync will make it easier for dataclass users to understand and use dataclass_transform and will simplify the maintenance of dataclass support in type checkers.

Additionally, we will consider adding dataclass_transform support in the future for features that have been adopted by multiple third-party libraries but are not supported by dataclasses.

Specification

The dataclass_transform decorator

This specification introduces a new decorator function in the typing module named dataclass_transform. This decorator can be applied to either a function that is itself a decorator, a class, or a metaclass. The presence of dataclass_transform tells a static type checker that the decorated function, class, or metaclass performs runtime "magic" that transforms a class, endowing it with dataclass-like behaviors.

If dataclass_transform is applied to a function, using the decorated function as a decorator is assumed to apply dataclass-like semantics. If the function has overloads, the dataclass_transform decorator can be applied to the implementation of the function or any one, but not more than one, of the overloads. When applied to an overload, the dataclass_transform decorator still impacts all usage of the function.

If dataclass_transform is applied to a class, dataclass-like semantics will be assumed for any class that directly or indirectly derives from the decorated class or uses the decorated class as a metaclass. Attributes on the decorated class and its base classes are not considered to be fields.

Examples of each approach are shown in the following sections. Each example creates a CustomerModel class with dataclass-like semantics. The implementation of the decorated objects is omitted for brevity, but we assume that they modify classes in the following ways:

* They synthesize an __init__ method using data fields declared within the class and its parent classes. * They synthesize __eq__ and __ne__ methods.

Type checkers supporting this PEP will recognize that the CustomerModel class can be instantiated using the synthesized __init__ method:

Decorator function example

Class example

Metaclass example

Decorator function and class/metaclass parameters

A decorator function, class, or metaclass that provides dataclass-like functionality may accept parameters that modify certain behaviors. This specification defines the following parameters that static type checkers must honor if they are used by a dataclass transform. Each of these parameters accepts a bool argument, and it must be possible for the bool value (True or False) to be statically evaluated.

* eq, order, frozen, init and unsafe_hash are parameters supported in the stdlib dataclass, with meanings defined in PEP 557. * kw_only, match_args and slots are parameters supported in the stdlib dataclass, first introduced in Python 3.10.

dataclass_transform parameters

Parameters to dataclass_transform allow for some basic customization of default behaviors:

* eq_default indicates whether the eq parameter is assumed to be True or False if it is omitted by the caller. If not specified, eq_default will default to True (the default assumption for dataclass). * order_default indicates whether the order parameter is assumed to be True or False if it is omitted by the caller. If not specified, order_default will default to False (the default assumption for dataclass). * kw_only_default indicates whether the kw_only parameter is assumed to be True or False if it is omitted by the caller. If not specified, kw_only_default will default to False (the default assumption for dataclass). * field_specifiers specifies a static list of supported classes that describe fields. Some libraries also supply functions to allocate instances of field specifiers, and those functions may also be specified in this tuple. If not specified, field_specifiers will default to an empty tuple (no field specifiers supported). The standard dataclass behavior supports only one type of field specifier called Field plus a helper function (field) that instantiates this class, so if we were

describing the stdlib dataclass behavior, we would provide the tuple argument (dataclasses.Field, dataclasses.field). * kwargs allows arbitrary additional keyword args to be passed to dataclass_transform. This gives type checkers the freedom to support experimental parameters without needing to wait for changes in typing.py. Type checkers should report errors for any unrecognized parameters.

In the future, we may add additional parameters to dataclass_transform as needed to support common behaviors in user code. These additions will be made after reaching consensus on typing-sig rather than via additional PEPs.

The following sections provide additional examples showing how these parameters are used.

Decorator function example

Class example

Metaclass example

Field specifiers

Most libraries that support dataclass-like semantics provide one or more "field specifier" types that allow a class definition to provide additional metadata about each field in the class. This metadata can describe, for example, default values, or indicate whether the field should be included in the synthesized __init__ method.

Field specifiers can be omitted in cases where additional metadata is not required:

Field specifier parameters

Libraries that support dataclass-like semantics and support field specifier classes typically use common parameter names to construct these field specifiers. This specification formalizes the names and meanings of the parameters that must be understood for static type checkers. These standardized parameters must be keyword-only.

These parameters are a superset of those supported by dataclasses.field, excluding those that do not have an impact on type checking such as compare and hash.

Field specifier classes are allowed to use other parameters in their constructors, and those parameters can be positional and may use other names.

* init is an optional bool parameter that indicates whether the field should be included in the synthesized __init__ method. If unspecified, init defaults to True. Field specifier functions can use overloads that implicitly specify the value of init using a literal bool value type (Literal[False] or Literal[True]). * default is an optional parameter that provides the default value for the field. * default_factory is an optional parameter that provides a runtime callback that returns the default value for the field. If neither default nor default_factory are specified, the field is assumed to have no default value and must be provided a value when the class is instantiated. * factory is an alias for default_factory. Stdlib dataclasses use the name default_factory, but attrs uses the name factory in many scenarios, so this alias is necessary for supporting attrs. * kw_only is an optional bool parameter that indicates whether the field should be marked as keyword-only. If true, the field will be keyword-only. If false, it will not be keyword-only. If unspecified, the value of the kw_only parameter on the object decorated with dataclass_transform will be used, or if that is unspecified, the value of kw_only_default on dataclass_transform will be used. * alias is an optional str parameter that provides an alternative name for the field. This alternative name is used in the synthesized __init__ method.

It is an error to specify more than one of default, default_factory and factory.

This example demonstrates the above:

Runtime behavior

At runtime, the dataclass_transform decorator's only effect is to set an attribute named __dataclass_transform__ on the decorated function or class to support introspection. The value of the attribute should be a dict mapping the names of the dataclass_transform parameters to their values.

For example:

Dataclass semantics

Except where stated otherwise in this PEP, classes impacted by dataclass_transform, either by inheriting from a class that is decorated with dataclass_transform or by being decorated with a function decorated with dataclass_transform, are assumed to behave like stdlib dataclass.

This includes, but is not limited to, the following semantics:

* Frozen dataclasses cannot inherit from non-frozen dataclasses. A class that has been decorated with dataclass_transform is considered neither frozen nor non-frozen, thus allowing frozen classes to inherit from it. Similarly, a class that directly specifies a metaclass that is decorated with dataclass_transform is considered neither frozen nor non-frozen.

Consider these class examples:

And these similar metaclass examples:

* Field ordering and inheritance is assumed to follow the rules specified in 557. This includes the effects of overrides (redefining a field in a child class that has already been defined in a parent class).

* PEP 557 indicates that all fields without default values must appear before fields with default values. Although not explicitly stated in PEP 557, this rule is ignored when init=False, and this specification likewise ignores this requirement in that situation. Likewise, there is no need to enforce this ordering when keyword-only parameters are used for __init__, so the rule is not enforced if kw_only semantics are in effect.

* As with dataclass, method synthesis is skipped if it would overwrite a method that is explicitly declared within the class. Method declarations on base classes do not cause method synthesis to be skipped.

```
        For example, if a class declares an __init__ method explicitly,
        an __init__ method will not be synthesized for that class.
```

* KW_ONLY sentinel values are supported as described in the Python docs and bpo-43532.

* ClassVar attributes are not considered dataclass fields and are ignored by dataclass mechanisms.

Undefined behavior

If multiple dataclass_transform decorators are found, either on a single function (including its overloads), a single class, or within a class hierarchy, the resulting behavior is undefined. Library authors should avoid these scenarios.

Reference Implementation

Pyright contains the reference implementation of type checker support for dataclass_transform. Pyright's dataClasses.ts source file would be a good starting point for understanding the implementation.

The attrs and pydantic libraries are using dataclass_transform and serve as real-world examples of its usage.

Rejected Ideas

auto_attribs parameter

The attrs library supports an auto_attribs parameter that indicates whether class members decorated with 526 variable annotations but with no assignment should be treated as data fields.

We considered supporting auto_attribs and a corresponding auto_attribs_default parameter, but decided against this because it is specific to attrs.

Django does not support declaring fields using type annotations only, so Django users who leverage dataclass_transform should be aware that they should always supply assigned values.

cmp parameter

The attrs library supports a bool parameter cmp that is equivalent to setting both eq and order to True. We chose not to support a cmp parameter, since it only applies to attrs. Users can emulate the cmp behaviour by using the eq and order parameter names instead.

Automatic field name aliasing

The attrs library performs automatic aliasing of field names that start with a single underscore, stripping the underscore from the name of the corresponding __init__ parameter.

This proposal omits that behavior since it is specific to attrs. Users can manually alias these fields using the alias parameter.

Alternate field ordering algorithms

The attrs library currently supports two approaches to ordering the fields within a class:

* Dataclass order: The same ordering used by dataclasses. This is the default behavior of the older APIs (e.g. attr.s). * Method Resolution Order (MRO): This is the default behavior of the newer APIs (e.g. define, mutable, frozen). Older APIs (e.g. attr.s) can opt into this behavior by specifying collect_by_mro=True.

The resulting field orderings can differ in certain diamond-shaped multiple inheritance scenarios.

For simplicity, this proposal does not support any field ordering other than that used by dataclasses.

Fields redeclared in subclasses

The attrs library differs from stdlib dataclasses in how it handles inherited fields that are redeclared in subclasses. The dataclass specification preserves the original order, but attrs defines a new order based on subclasses.

For simplicity, we chose to only support the dataclass behavior. Users of attrs who rely on the attrs-specific ordering will not see the expected order of parameters in the synthesized __init__ method.

Django primary and foreign keys

Django applies additional logic for primary and foreign keys. For example, it automatically adds an id field (and __init__ parameter) if there is no field designated as a primary key.

As this is not broadly applicable to dataclass libraries, this additional logic is not accommodated with this proposal, so users of Django would need to explicitly declare the id field.

Class-wide default values

SQLAlchemy requested that we expose a way to specify that the default value of all fields in the transformed class is None. It is typical that all SQLAlchemy fields are optional, and None indicates that the field is not set.

We chose not to support this feature, since it is specific to SQLAlchemy. Users can manually set default=None on these fields instead.

Descriptor-typed field support

We considered adding a boolean parameter on dataclass_transform to enable better support for fields with descriptor types, which is common in SQLAlchemy. When enabled, the type of each parameter on the synthesized __init__ method corresponding to a descriptor-typed field would be the type of the value parameter to the descriptor's __set__ method rather than the descriptor type itself. Similarly, when setting the field, the __set__ value type would be expected. And when getting the value of the field, its type would be expected to match the return type of __get__.

This idea was based on the belief that dataclass did not properly support descriptor-typed fields. In fact it does, but type checkers (at least mypy and pyright) did not reflect the runtime behavior which led to our misunderstanding. For more details, see the Pyright bug.

converter field specifier parameter

The attrs library supports a converter field specifier parameter, which is a Callable that is called by the generated __init__ method to convert the supplied value to some other desired value. This is tricky to support since the parameter type in the synthesized __init__ method needs to accept uncovered values, but the resulting field is typed according to the output of the converter.

Some aspects of this issue are detailed in a Pyright discussion.

There may be no good way to support this because there's not enough information to derive the type of the input parameter. One possible solution would be to add support for a converter field specifier parameter but then use the Any type for the corresponding parameter in the __init__ method.

References

Copyright

This document is placed in the public domain or under the CC0-1.0-Universal license, whichever is more permissive.